

MindTrace Diagrams Explanation

1. The Architecture Diagram

The Architecture Diagram outlines the structural layers of the MindTrace application. It separates the system into four major blocks: the User, Frontend, Backend, and Database.

--- User (Browser + Webcam)

- What it is: The end-user interacting with the MindTrace application through a web browser on their device.
- Role in detail: The user accesses the application and grants permission to use their webcam. The browser captures the live video feed (frames) and sends the user's inputs (like button clicks) to the frontend. It is the starting point for all interactions and data generation.

--- Frontend (Hosted on Vercel)

This is the client-facing side of the application, responsible for rendering the user interface and handling client-side logic.

- React + TypeScript App: The core framework used to build the user interface. React allows for dynamic, interactive, and responsive UI components, while TypeScript ensures type safety, making the codebase more reliable and easier to maintain.
- Pages / Views:
 - Live Session: The primary interface where the user takes a session. It displays the live webcam feed with an overlay showing real-time emotion and focus metrics.
 - Dashboard: A central hub showing an overview of the user's performance, recent sessions, and aggregated statistics.
 - History & Reports: Detailed logs of past sessions, allowing the user to review trends over time through charts and graphs.
 - Alerts: Notifications triggered by specific events (e.g., if the user's focus drops significantly for an extended period).

--- Backend (Hosted on Hugging Face Spaces)

This is the computational engine of the application. It processes requests, handles heavy machine learning inference, and manages business logic.

- FastAPI (REST API): A modern, fast web framework for building APIs in Python. It acts as the gateway for the backend, receiving HTTP requests (like sending frames or fetching history) from the frontend and returning the appropriate responses.
- ML Pipeline (ResNet18 + MediaPipe): The core artificial intelligence engine.

- *MediaPipe* is used first for highly efficient Face Detection and extracting face landmarks (FaceMesh).
- *ResNet18* (a deep convolutional neural network) then takes the cropped face image and predicts the user's current emotion and calculates a focus/engagement score based on head pose and facial expressions.
- Auth (JWT Tokens): The security layer. It uses JSON Web Tokens (JWT) to securely authenticate users. When a user logs in, they receive a token that they must attach to subsequent requests to prove they are authorized to access the system.

--- Database (MongoDB Atlas)

The persistent storage layer. MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. Atlas is the cloud-hosted version of MongoDB.

- Collections:

- users: Stores user credentials (securely hashed passwords), profile details, and roles.
- sessions: Keeps track of individual tracking sessions (start time, end time, total duration, overall average focus score).
- emotions: Stores the granular, second-by-second emotion predictions and focus scores tied to a specific session.
- events: Logs specific system or user events (like alerts triggered or session interruptions).

2. The Live Session Flow Diagram

The Flow Diagram details the chronological sequence of events that happen when a user actually uses the application to monitor their focus.

--- Phase 1: Starting the Session

1. User Action: The user clicks the "Start Session" button on the React frontend.
2. API Call: The Frontend sends a `GET /start` HTTP request to the FastAPI Backend.
3. Database Creation: The Backend communicates with MongoDB to create a brand new, empty session record in the `sessions` collection.
4. Confirmation: The Backend responds to the Frontend with a unique `session\_id`, confirming that the session is officially active.

--- Phase 2: The Continuous Loop (Every 1 Second)

Once the session is active, the system enters a continuous loop that repeats every second until the session is stopped.

1. Frame Capture: The Frontend locally grabs a single image frame from the user's webcam feed.
2. Data Transmission: The Frontend sends this frame over the network to the Backend via a `POST /analyze\_frame` request (attaching the `session\_id`).
3. Processing (Detect ? Predict): The Backend feeds the image into the ML Pipeline. MediaPipe detects the face, and ResNet18 predicts the emotion and calculates the focus score.
4. Returning Results: The Backend instantly replies to the Frontend with the calculated `emotion` (e.g., "Focused", "Distracted") and a numerical `focus score`.
5. UI Update: The Frontend updates the screen, drawing a bounding box or overlay on the user's video feed and updating live charts with the new data.

--- Phase 3: Stopping the Session

1. User Action: The user clicks the "Stop" button.
2. API Call: The Frontend sends a `GET /stop` request to the Backend.
3. Data Finalization: The Backend takes all the accumulated emotion data from that session and permanently saves it into the MongoDB `emotions` and `sessions` collections, marking the session as "ended".
4. Confirmation: The Backend sends a "Done" signal to the Frontend, which then typically redirects the user to a summary report of the session they just completed.